

Program

Webnovel Hub — Requirements

SCRAPER | SOURCE MANAGEMENT

[Scraper][Source Management] 0001 : The system must support multiple scraping adapters, one per supported source site, each implementing a common interface (getMetadata, getChapterList, getChapterContent).

[Scraper][Source Management] 0001-01 : The system must include a scraping adapter for Ranobes.

[Scraper][Source Management] 0001-02 : The system must include a scraping adapter for WTR Lab.

[Scraper][Source Management] 0001-03 : The system must include a scraping adapter for Royal Road.

[Scraper][Source Management] 0001-04 : The system must be able to detect which supported source site a given URL belongs to and route it to the correct adapter automatically.

[Scraper][Source Management] 0001-05 : If a URL does not match any supported source, the system must return a clear error indicating the site is not supported, without crashing.

SCRAPER | NOVEL INGESTION

[Scraper][Ingestion] 0002 : When a user submits a novel URL, the system must scrape and store the novel's metadata before returning a response.

[Scraper][Ingestion] 0002-01 : Scraped novel metadata must include: title, author, cover image URL, description, tags/genres, source URL, and source site identifier.

[Scraper][Ingestion] 0002-02 : The system must scrape the full chapter list of a novel upon ingestion, storing each chapter's title, chapter number, and source URL.

[Scraper][Ingestion] 0002-03 : Chapter content must NOT be scraped at ingestion time. Content is fetched only when a user opens a chapter for the first time (lazy loading).

[Scraper][Ingestion] 0002-04 : Once a chapter's content has been fetched and stored locally, subsequent reads must be served from the local database, not re-scraped.

[Scraper][Ingestion] 0002-05 : If a novel URL has already been added to the library, the system must notify the user and must not create a duplicate entry.

[Scraper][Ingestion] 0002-06 : Cover images must be downloaded and stored locally, not hotlinked from source sites, to prevent broken images if the source changes.

[Scraper][Ingestion] 0002-07 : When a user opens a chapter, the system must automatically trigger background pre-fetching for the next 3 chapters ahead (configurable), storing their content silently so they are available instantly when the user navigates forward. Pre-fetching must not block or delay the rendering of the currently requested chapter.

[Scraper][Ingestion] 0002-08 : When the system loads a novel, the system must automatically trigger background pre-fetching for the first 3 chapters (configurable)

SCRAPER | UPDATE JOB

[Scraper][Update Job] 0003 : The system must run a background job that periodically checks all followed novels for new chapters.

[Scraper][Update Job] 0003-01 : The update job must run on a configurable interval, defaulting to once every 6 hours.

[Scraper][Update Job] 0003-02 : The update job must only check novels whose status is set to "Reading" or "Following". Completed, On Hold, and Dropped novels must be excluded.

[Scraper][Update Job] 0003-03 : When new chapters are detected, the system must store their metadata (title, number, source URL) and flag them as unread and new.

[Scraper][Update Job] 0003-04 : The update job must log each run, recording timestamp, novels checked, new chapters found, and any errors encountered per novel.

[Scraper][Update Job] 0003-05 : If a novel's source page returns an error or unexpected structure during an update check, that novel must be flagged as "update failed" and the job must continue checking remaining novels without stopping.

[Scraper][Update Job] 0003-06 : The system must expose a manual "Check for updates" trigger per novel, which runs the same logic as the scheduled job but for a single novel on demand.

SCRAPER | FAILURE HANDLING

[Scraper][Failure Handling] 0004 : The system must handle scraping failures without crashing, categorizing errors as: network error, structure changed, rate limited, or unknown.

[Scraper][Failure Handling] 0004-01 : If a scrape fails, the system must retry up to 3 times with an exponential backoff delay before marking the operation as failed.

[Scraper][Failure Handling] 0004-02 : Failed ingestion attempts must not leave partial novel records in the database. The operation must be fully rolled back on failure.

[Scraper][Failure Handling] 0004-03 : The user must be able to see the failure reason for any novel or chapter that could not be scraped, in plain language.

LIBRARY | COLLECTION

[Library][Collection] 0005 : The library must display all added novels as a grid of cards, each showing cover image, title, author, source site, reading status, and a new chapter badge when applicable.

[Library][Collection] 0005-01 : Each novel in the library must have one of the following statuses, user-assignable: Reading, Following, On Hold, Dropped, Completed.

[Library][Collection] 0005-02 : Novels with unread new chapters must be visually distinguished in the library view with a badge showing the count of new chapters.

[Library][Collection] 0005-03 : The library must display the user's reading progress per novel (e.g. "Chapter 143 / 289").

[Library][Collection] 0005-04 : The user must be able to mark a novel as a favorite, and favorites must be visually distinguished in the library.

LIBRARY | FILTERING AND SORTING

[Library][Filters] 0006 : The library must support filtering novels by reading status (Reading, Following, On Hold, Dropped, Completed, or All).

[Library][Filters] 0006-01 : The library must support filtering by source site (Ranobes, WTR Lab, Royal Road, or All).

[Library][Filters] 0006-02 : The library must support filtering to show only favorites.

[Library][Filters] 0006-03 : The library must support filtering to show only novels with unread new chapters.

[Library][Filters] 0006-04 : The library must support sorting by: last read date, last updated date, title (A-Z), date added, and reading progress percentage.

[Library][Filters] 0006-05 : The library must support a text search bar that filters novels by title or author in real time.

[Library][Filters] 0006-06 : Active filters and sort selection must persist across page navigation within the same session.

LIBRARY | NOVEL DETAIL PAGE

[Library][Novel Detail] 0007 : Each novel must have a dedicated detail page showing all metadata, full description, chapter list, reading status, and reading progress.

[Library][Novel Detail] 0007-01 : The novel detail page must show the full chapter list with each chapter's title, number, read/unread state, and a "new" indicator for recently added chapters.

[Library][Novel Detail] 0007-02 : The chapter list must support pagination or virtual scrolling for novels with large chapter counts (500+).

[Library][Novel Detail] 0007-03 : The novel detail page must include a prominent "Continue Reading" button that navigates directly to the last read chapter.

[Library][Novel Detail] 0007-04 : The user must be able to change the novel's reading status directly from the detail page.

[Library][Novel Detail] 0007-05 : The user must be able to delete a novel from their library from the detail page, with a confirmation prompt. Deletion must remove all associated chapter content from the database.

READER | CORE READING EXPERIENCE

[Reader][Core] 0008 : The reader must display chapter content in a clean, distraction-free view with no site navigation visible by default.

[Reader][Core] 0008-01 : If a chapter's content has not yet been fetched, the reader must trigger a scrape on demand, show a loading state, and display the content once available.

[Reader][Core] 0008-02 : The reader must provide previous chapter and next chapter navigation controls, accessible via buttons and keyboard arrow keys.

[Reader][Core] 0008-03 : When a user opens a chapter, it must automatically be marked as read and the novel's reading progress must be updated to that chapter.

[Reader][Core] 0008-04 : When a user reaches the end of the last available chapter, the system must display a message indicating no further chapters are available, instead of a broken navigation link.

READER | SETTINGS [UI]

[Reader][Settings][UI] 0009 : The reader must provide a settings panel accessible from within the reading view without leaving the page.

[Reader][Settings][UI] 0009-01 : The reader must support at least three color themes: Light, Dark, and Sepia.

[Reader][Settings][UI] 0009-02 : The reader must allow the user to adjust font size within a defined range (e.g. 14px – 28px) in increments.

[Reader][Settings][UI] 0009-03 : The reader must allow the user to adjust line spacing (e.g. 1.4 – 2.2) in increments.

[Reader][Settings][UI] 0009-04 : The reader must allow the user to adjust content column width (e.g. narrow / medium / wide).

[Reader][Settings][UI] 0009-05 : All reader settings must persist across sessions using local storage or a user preferences table in the database.

READER | PROGRESS SYNC [Mobile]

[Reader][Progress][Mobile] 0010 : Reading progress (last chapter read) must be stored server-side so it is consistent across desktop and mobile browsers.

[Reader][Progress][Mobile] 0010-01 : If the user opens a novel on a different device, the "Continue Reading" button must reflect the most recently updated progress from the server, not the local browser state.

OFFLINE | STORAGE

[Offline][Storage] 0011 : The system must allow the user to mark individual novels for offline availability.

[Offline][Storage] 0011-01 : When a novel is marked for offline use, all already-fetched chapter content must be cached locally on the device via a service worker or equivalent mechanism.

[Offline][Storage] 0011-02 : Chapters fetched while online must be automatically added to the offline cache if the novel is marked for offline use.

[Offline][Storage] 0011-03 : The user must be able to see, per novel, how many chapters are available offline vs. total chapters stored.

[Offline][Storage] 0011-04 : The user must be able to clear the offline cache for a specific novel without removing it from the library.

SYSTEM | API DESIGN

[System][API] 0012 : The backend must expose a RESTful API consumed by the frontend. No server-side rendering.

[System][API] 0012-01 : All API endpoints must return consistent JSON response envelopes with at minimum: success (bool), data (object or null), and error (string or null).

[System][API] 0012-02 : All API errors must return semantically correct HTTP status codes (400 for bad input, 404 for not found, 500 for server errors, etc.).

[System][API] 0012-03 : Long-running operations (novel ingestion, manual update check) must be handled asynchronously. The API must return immediately with a job ID, and the client must poll or receive a status update when the job completes.

SYSTEM | DATA PERSISTENCE

[System][Persistence] 0013 : All novel metadata, chapter metadata, chapter content, reading progress, and user preferences must be stored in a relational database (MySQL).

[System][Persistence] 0013-01 : Chapter content must be stored as raw text in the database, stripped of source site HTML chrome (ads, navigation, comments) but preserving paragraph structure.

[System][Persistence] 0013-02 : The database schema must use proper foreign keys and cascade rules so that deleting a novel also deletes all its associated chapters and progress records.